MOBİL PROGRAMLAMA — LAB FÖYÜ 2

MOBİL KULLANICI ARAYÜZÜ TASARIMI VE YERLEŞİM

Laboratuvar Uygulaması: Teknik Destek Yönetim Paneli (IT Help Desk Dashboard)

120 dakika
Orta–Zor Seviye
React Native + Expo
JavaScript

1. Laboratuvarın Amacı

Bu laboratuvar çalışmasında öğrenciler, React Native kullanarak profesyonel bir mobil yönetim paneli tasarlayacaklardır.

Önceki laboratuvarda kullanıcıların teknik arıza bildiriminde bulunabilecekleri bir form geliştirilmişti. Bu hafta ise oluşturulan destek taleplerinin teknik destek personeli tarafından görüntülenebildiği, filtrelenebildiği ve durumlarının güncellenebildiği bir yönetim ekranı geliştirilecektir.

Bu çalışmanın temel odağı; mobil arayüz yerleşimi, Flexbox düzeni, yeniden kullanılabilir kart bileşenleri, listeleme ve farklı ekran boyutlarına uyumdur.

1.1. Öğrenme kazanımları

Laboratuvar sonunda öğrenci:

1. React Native'de Flexbox yerleşim sistemini kullanabilir.
2. `flexDirection`, `justifyContent`, `alignItems`, `flex` ve `gap` özelliklerini uygulayabilir.
3. Bileşenleri satır ve sütun düzenlerinde konumlandırabilir.
4. Özet bilgi kartları ve liste elemanları tasarlayabilir.
5. `FlatList` kullanarak veri listelerini görüntüleyebilir.
6. Tekrar kullanılabilir React bileşenleri geliştirebilir.
7. `useState` kullanarak liste filtreleme ve durum güncelleme işlemlerini gerçekleştirebilir.
8. `useWindowDimensions` ile ekran genişliğine duyarlı yerleşimler oluşturabilir.
9. Renk, boşluk, hizalama ve görsel hiyerarşi ilkelerini kullanabilir.
10. Mobil yönetim paneline yönelik temel kullanıcı deneyimi gereksinimlerini karşılayabilir.

2. Gerçek Hayat Problemi ve Uygulama Senaryosu

Senaryo — Müşteriden Gelen İkinci Geliştirme Talebi

Geçen hafta geliştirdiğiniz Teknik Destek ve Arıza Bildirim uygulaması, üniversite personelinin destek talebi oluşturabilmesini sağlamıştır.

Ancak Bilgi İşlem Daire Başkanlığı, gelen talepleri tek ekranda izleyebileceği bir yönetim paneline de ihtiyaç duymaktadır.

Destek personeli, hangi arızaların beklediğini, hangilerinin işlemde olduğunu ve hangi taleplerin tamamlandığını kolayca görememektedir.

Müşteri sizden ikinci bir mobil ekran geliştirmenizi istemiştir.

Göreviniz: Teknik destek taleplerini özet kartlarıyla sunan, durumlarına göre filtreleyen, arızaları listeleyen ve durumlarının değiştirilmesini sağlayan bir mobil yönetim paneli geliştirmektir.

2.1. Müşteri gereksinimleri

No	Gereksinim
G1	Ekranın üstünde yönetim paneli başlığı bulunmalıdır.
G2	Toplam, Açık, İşlemde ve Tamamlanan talep sayıları gösterilmelidir.
G3	Özet bilgiler görsel kartlar içerisinde sunulmalıdır.
G4	Talepler kaydırılabilir bir listede gösterilmelidir.
G5	Her talep kartında başlık, kategori, birim, öncelik ve durum bulunmalıdır.
G6	Talep durumları farklı renklerle gösterilmelidir.
G7	Talepler durumlarına göre filtrelenebilmelidir.
G8	Kullanıcı talep durumunu değiştirebilmelidir.
G9	Özet kartları durum değişikliklerine göre güncellenmelidir.
G10	Arayüz farklı ekran genişliklerine uyum sağlamalıdır.

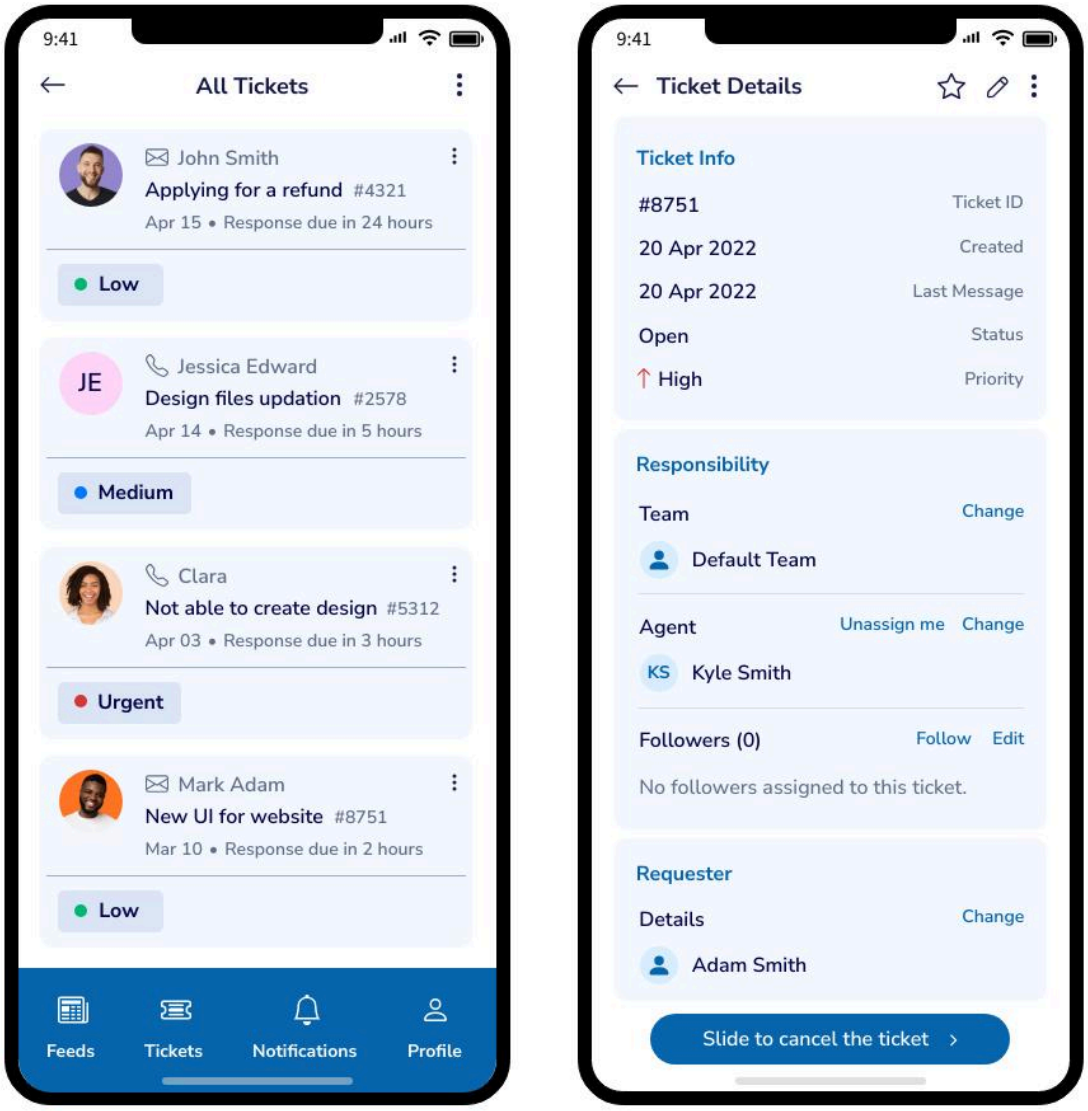
Kapsam: Bu laboratuvarda önceki haftanın iş senaryosu kullanılacak, ancak proje sıfırdan oluşturulacaktır. Talepler, uygulamanın içerisinde tanımlanan örnek verilerden oluşacaktır. Veritabanı, API ve ekranlar arası navigasyon kullanılmayacaktır.

3. Laboratuvar Süre Planı

Aşama	İçerik	Süre
1	Proje oluşturma ve çalıştırma	10 dk

2	Flexbox ve yerleşim özellikleri	20 dk
3	Yönetim paneli başlığı ve özet kartları	20 dk
4	Tekrar kullanılabilir talep kartı	20 dk
5	FlatList ve filtreleme	20 dk
6	Durum güncelleme ve responsive tasarım	15 dk
7	Bağımsız geliştirme ve testler	15 dk
Toplam		120 dk

4. Geliştirilecek Uygulamanın Arayüzü



Hedef arayüz için temsili tasarım referansı. Geliştirilecek uygulama; özet kartları, filtreler ve teknik destek talep kartlarını içerecektir.

Panelin genel yapısı şu şekilde planlanmıştır:

BİLGİ İŞLEM DAİRE BAŞKANLIĞI

Teknik Destek Paneli

Destek taleplerini buradan yönetin.

Toplam Talep

3

Açık Talep

3

İşlemde

2

Tamamlanan

Tümü
Açık
İşlemde
Tamamlanan

Bilgisayar açılmıyor

Yüksek

Bilgisayar · Öğrenci İşleri

Açık

TLP-1001

İşleme Al

Yazıcı bağlantı sorunu

Orta

Yazıcı · Personel Daire Başkanlığı

İşlemde

TLP-1002

Tamamla

Yukarıdaki şema arayüz bölümlerini açıklamak için hazırlanmıştır. Gerçek uygulamada kartlar örnek veri dizisinden üretilecek, filtre ve durum butonları çalışacaktır.

5. Projenin Sıfırdan Oluşturulması

Adım 1 — Yeni proje klasörünü oluşturunuz

Bilgisayarınızda aşağıdaki klasörü oluşturunuz:

`MobilProgramlama/Lab4-TeknikDestekPaneli`

Visual Studio Code programını açınız ve bu klasörü File → Open Folder üzerinden seçiniz.

Ardından Terminal → New Terminal seçeneğiyle terminali açınız.

Adım 2 — Expo projesini oluşturunuz

```
npx create-expo-app@latest --template blank ./
```

Bu komut, yönlendirme altyapısı olmadan temel bir JavaScript Expo projesi oluşturur.

Adım 3 — Safe Area paketini kurunuz

```
npx expo install react-native-safe-area-context
```

Bu paket, içeriğin ekran çentiği, durum çubuğu veya sistem gezinme alanı tarafından kapatılmasını engellemek için kullanılacaktır.

Adım 4 — Uygulamayı çalıştırınız

```
npx expo start
```

Bağlantı sorunu yaşanması durumunda:

```
npx expo start --tunnel
```

Proje başarıyla oluşturulduğunda `App.js` dosyasını açınız.

6. Mobil Arayüz Yerleşimini Öğrenelim — Flexbox

React Native'de mobil arayüzlerin düzenlenmesinde Flexbox yerleşim modeli kullanılır.

Flexbox, bileşenlerin ekranda hangi yönde sıralanacağını, nasıl hizalanacağını ve kullanılabilir alanı nasıl paylaşacağını belirlememizi sağlar.

6.1. flexDirection

Bileşenlerin satır veya sütun şeklinde sıralanmasını belirler.

```
<View style={{ flexDirection: 'row' }}>
  <Text>Toplam</Text>
  <Text>Açık</Text>
```

```
  <Text>Tamamlanan</Text>
</View>
```

Önemli: React Native'de varsayılan `flexDirection` değeri `column` şeklindedir.

6.2. justifyContent

Ana eksen üzerindeki hizalamayı kontrol eder.

`flexDirection: 'row'` olduğunda yatay hizalamayı, `column` olduğunda dikey hizalamayı etkiler.

Örnek:

```
<View style={{
  flexDirection: 'row',
  justifyContent: 'space-between'
}}>
  <Text>Talep No</Text>
  <Text>Durum</Text>
</View>
```

Bu yaklaşım talep kartının iki ucunda farklı bilgilerin gösterilmesini sağlar.

6.3. alignItems

Bileşenlerin çapraz eksende nasıl hizalanacağını belirler.

```
<View style={{
  flexDirection: 'row',
  justifyContent: 'space-between',
  alignItems: 'center'
}}>
  <Text>Arıza Başlığı</Text>
  <Text>Yüksek Öncelik</Text>
</View>
```

6.4. flex

`flex`, kullanılabilir alanın bileşenler arasında paylaşılmasını sağlar.

```
<View style={{ flexDirection: 'row', gap: 10 }}>
  <View style={{
    flex: 1,
    backgroundColor: '#bfdbfe',
    height: 70
  }} />
```

```
  <View style={{
    flex: 1,
    backgroundColor: '#bbf7d0',
    height: 70
  }} />
</View>
```

İki bileşen de `flex: 1` kullandığından kullanılabilir genişliği eşit paylaşır.

6.5. flexWrap ve gap

`flexWrap`, öğelerin bir satıra sığmadığında alt satıra geçmesini sağlar.

`gap`, bileşenler arasındaki boşluğu belirler.

```
<View style={{
  flexDirection: 'row',
  flexWrap: 'wrap',
  gap: 10
}}>
  <Text>Toplam</Text>
  <Text>Açık</Text>
  <Text>İşlemde</Text>
  <Text>Tamamlanan</Text>
</View>
```

Bu özellikleri yönetim panelinin özet kartlarında ve filtre seçeneklerinde kullanacağız.

6.6. Öğrenci alıştırması — İki kartlı yerleşim

`App.js` dosyasını geçici olarak aşağıdaki kodla değiştiriniz:

```
import React from 'react';
import { View, Text, StyleSheet } from 'react-native';
import {
  SafeAreaProvider,
  SafeAreaView
} from 'react-native-safe-area-context';

function OrnekEkran() {
  return (
    <SafeAreaView style={styles.container}>
      <View style={styles.row}>
        <View style={styles.card}>
          <Text>Toplam Talep</Text>
          <Text style={styles.number}>8</Text>
        </View>
```

```
        <View style={styles.card}>
          <Text>Açık Talep</Text>
          <Text style={styles.number}>3</Text>
        </View>
      </View>
    </SafeAreaView>
  );
}

export default function App() {
  return (
    <SafeAreaProvider>
      <OrnekEkran />
    </SafeAreaProvider>
  );
}

const styles = StyleSheet.create({
  container: {
    flex: 1,
    padding: 20,
    backgroundColor: '#f1f5f9'
  },
  row: {
    flexDirection: 'row',
    gap: 12
  },
  card: {
    flex: 1,
    padding: 20,
    borderRadius: 12,
    backgroundColor: 'white'
  },
  number: {
    fontSize: 28,
    fontWeight: 'bold',
    marginTop: 10
  }
});
```

Uygulama: Önce iki kartın yan yana görüntülendiğini kontrol ediniz. Ardından `flexDirection: 'row'` değerini `column` olarak değiştiriniz ve ekran düzenindeki değişimi gözlemleyiniz.

Alıştırmayı tamamladıktan sonra bir sonraki bölümdeki yönetim paneli uygulamasına geçiniz.

7. Yönetim Panelinin Adım Adım Geliştirilmesi

Adım 1 — Temel ekran yapısını hazırlayınız

`App.js` dosyasının içeriğini aşağıdaki kodla değiştiriniz:

```
import React, { useState } from 'react';

import {
  View,
  Text,
  Pressable,
  FlatList,
  StyleSheet,
  StatusBar,
  useWindowDimensions
} from 'react-native';

import {
  SafeAreaProvider,
  SafeAreaView
} from 'react-native-safe-area-context';

function TeknikDestekPaneli() {
  return (
    <SafeAreaView style={styles.container}>
      <StatusBar
        barStyle="dark-content"
        backgroundColor="#f1f5f9"
      />

      <View style={styles.header}>
        <Text style={styles.headerLabel}>
          BİLGİ İŞLEM
        </Text>

        <Text style={styles.title}>
          Teknik Destek Paneli
        </Text>

        <Text style={styles.subtitle}>
          Destek taleplerini buradan yönetin.
        </Text>
      </View>
    </SafeAreaView>
  );
}

export default function App() {
  return (
```

```
    <SafeAreaProvider>
      <TeknikDestekPaneli />
    </SafeAreaProvider>
  );
}

const styles = StyleSheet.create({
  container: {
    flex: 1,
    backgroundColor: '#f1f5f9'
  },
  header: {
    padding: 20,
    backgroundColor: '#dbeafe'
  },
  headerLabel: {
    fontSize: 12,
    fontWeight: 'bold',
    color: '#1d4ed8'
  },
  title: {
    fontSize: 25,
    fontWeight: 'bold',
    color: '#0f172a',
    marginTop: 6
  },
  subtitle: {
    fontSize: 14,
    color: '#475569',
    marginTop: 6
  }
});
```

Kontrol: Yönetim panelinin başlığı ekranın üst kısmında görünmelidir.

Adım 2 — Örnek talep verilerini oluşturunuz

Gerçek bir iş uygulamasında bu veriler çoğunlukla sunucudan alınır. Bu hafta kullanıcı arayüzü geliştirmeye odaklandığımız için verileri JavaScript dizisi içerisinde tanımlayacağız.

`App` fonksiyonunun üstüne aşağıdaki diziyi ekleyiniz:

```
const baslangicTalepleri = [
  {
    id: 'TLP-1001',
    baslik: 'Bilgisayar açılmıyor',
    kategori: 'Bilgisayar',
    birim: 'Öğrenci İşleri',
```

```
  oncelik: 'Yüksek',
  durum: 'Açık'
},
{
  id: 'TLP-1002',
  baslik: 'Yazıcı bağlantı sorunu',
  kategori: 'Yazıcı',
  birim: 'Personel Daire Başkanlığı',
  oncelik: 'Orta',
  durum: 'İşlemde'
},
{
  id: 'TLP-1003',
  baslik: 'İnternet bağlantısı kesildi',
  kategori: 'İnternet',
  birim: 'Kütüphane',
  oncelik: 'Yüksek',
  durum: 'Açık'
},
{
  id: 'TLP-1004',
  baslik: 'Ofis yazılımı hata veriyor',
  kategori: 'Yazılım',
  birim: 'Muhasebe',
  oncelik: 'Düşük',
  durum: 'Tamamlandı'
},
{
  id: 'TLP-1005',
  baslik: 'Bilgisayar çok yavaş',
  kategori: 'Bilgisayar',
  birim: 'Mühendislik Fakültesi',
  oncelik: 'Orta',
  durum: 'İşlemde'
},
{
  id: 'TLP-1006',
  baslik: 'Yazıcı kağıt sıkıştırıyor',
  kategori: 'Yazıcı',
  birim: 'Rektörlük',
  oncelik: 'Düşük',
  durum: 'Tamamlandı'
},
{
  id: 'TLP-1007',
  baslik: 'Kablosuz ağ çalışmıyor',
  kategori: 'İnternet',
  birim: 'Merkezi Derslikler',
```

```
    oncelik: 'Yüksek',
    durum: 'Açık'
  },
  {
    id: 'TLP-1008',
    baslik: 'Program lisans sorunu',
    kategori: 'Yazılım',
    birim: 'Bilgi İşlem',
    oncelik: 'Orta',
    durum: 'İşlemde'
  }
];
```

Veri dizisini incelediğinizde her talebin aynı alanlara sahip olduğunu göreceksiniz.

Düşünme sorusu: Bu verileri birbirinden bağımsız sekiz değişkende tutmak yerine bir nesne dizisinde tutmanın avantajı nedir?

Adım 3 — Talep durumunu yönetiniz

`TeknikDestekPaneli` fonksiyonu içerisinde `return` ifadesinden önce aşağıdaki durumları ekleyiniz:

```
const [talepler, setTalepler] = useState(
  baslangicTalepleri
);

const [aktifFiltre, setAktifFiltre] = useState('Tümü');
```

Artık `talepler` dizisi uygulama çalışırken güncellenebilecektir.

Adım 4 — Talep sayılarını hesaplayınız

Aşağıdaki kodu durum değişkenlerinin altına ekleyiniz:

```
const toplam = talepler.length;

const acik = talepler.filter(
  item => item.durum === 'Açık'
).length;

const islemde = talepler.filter(
  item => item.durum === 'İşlemde'
).length;

const tamamlanan = talepler.filter(
  item => item.durum === 'Tamamlandı'
).length;
```

Bu kod `filter()` fonksiyonunu kullanarak belirli koşulu sağlayan kayıtları sayar.

Örneğin:

item.durum === 'Açık'

koşulu yalnızca açık durumdaki kayıtları belirler.

Başlangıç verilerine göre:

Toplam	Açık	İşlemde	Tamamlanan
8	3	3	2

Adım 5 — Özet kartı bileşeni oluşturunuz

Aynı görünümde dört ayrı kart oluşturmak yerine, parametre alabilen tek bir bileşen geliştireceğiz.

Bu yaklaşım yeniden kullanılabilir bileşen (reusable component) olarak adlandırılır.

`TeknikDestekPaneli` fonksiyonundan önce aşağıdaki bileşeni oluşturunuz:

```
function OzetKarti({ baslik, sayi, renk }) {
  return (
    <View style={styles.summaryCard}>
      <Text style={[styles.summaryNumber, { color: renk }]}>
        {sayi}
      </Text>

      <Text style={styles.summaryLabel}>
        {baslik}
      </Text>
    </View>
  );
}
```

Ardından `header` bölümünün altına geçici olarak şu kartları ekleyiniz:

```
<View style={styles.summaryGrid}>
  <OzetKarti
    baslik="Toplam Talep"
    sayi={toplam}
    renk="#2563eb"
  />

  <OzetKarti
```

```
    baslik="Açık Talep"
    sayi={acik}
    renk="#dc2626"
  />

  <OzetKarti
    baslik="İşlemde"
    sayi={islemde}
    renk="#d97706"
  />

  <OzetKarti
    baslik="Tamamlanan"
    sayi={tamamlanan}
    renk="#16a34a"
  />
</View>
```

Stiller:

```
summaryGrid: {
  flexDirection: 'row',
  flexWrap: 'wrap',
  padding: 15,
  gap: 10
},

summaryCard: {
  width: '47%',
  padding: 18,
  backgroundColor: 'white',
  borderRadius: 12
},

summaryNumber: {
  fontSize: 28,
  fontWeight: 'bold'
},

summaryLabel: {
  color: '#64748b',
  fontSize: 13,
  marginTop: 6
}
```

Kontrol: Dört özet kartı iki satır ve iki sütun şeklinde görünmelidir.

Not: Bu aşamada yüzde tabanlı genişlik başlangıç örneği olarak kullanılmaktadır. İlerleyen adımda ekran genişliğine göre daha güvenilir, responsive bir kart yerleşimi oluşturacağız.

Adım 6 — Talep kartı bileşenini geliştiriniz

Yönetim panelinde her arıza talebi bir kart içerisinde görüntülenecektir.

Her kartta aşağıdaki bilgiler yer alacaktır:

- Talep numarası
- Arıza başlığı
- Kategori ve birim
- Öncelik seviyesi
- Talep durumu
- Durum güncelleme butonu

Önce öncelik ve durum renklerini tanımlayalım.

`App.js` içerisinde, örnek veri dizisinin altına aşağıdaki fonksiyonları ekleyiniz:

```
const oncelikRengi = (oncelik) => {
  if (oncelik === 'Yüksek') return '#dc2626';
  if (oncelik === 'Orta') return '#d97706';
  return '#16a34a';
};

const durumRengi = (durum) => {
  if (durum === 'Açık') return '#dc2626';
  if (durum === 'İşlemde') return '#2563eb';
  return '#16a34a';
};
```

Ardından `OzetKarti` bileşeninin altına şu bileşeni oluşturunuz:

```
function TalepKarti({ talep, onDurumDegistir }) {
  return (
    <View style={styles.ticketCard}>

      <View style={styles.ticketTop}>
        <Text style={styles.ticketId}>
          {talep.id}
        </Text>

        <Text
          style={{
            color: oncelikRengi(talep.oncelik),
            fontWeight: 'bold'
          }}
        >
```

```
          {talep.oncelik}
        </Text>
      </View>

      <Text style={styles.ticketTitle}>
        {talep.baslik}
      </Text>

      <Text style={styles.ticketDetail}>
        {talep.kategori} • {talep.birim}
      </Text>

      <View style={styles.ticketBottom}>

        <Text
          style={[
            styles.statusText,
            { color: durumRengi(talep.durum) }
          ]}
        >
          {talep.durum}
        </Text>

        {talep.durum !== 'Tamamlandı' && (
          <Pressable
            style={styles.actionButton}
            onPress={() => onDurumDegistir(talep.id)}
          >
            <Text style={styles.actionText}>
              {talep.durum === 'Açık'
                ? 'İşleme Al'
                : 'Tamamla'}
            </Text>
          </Pressable>
        )}

      </View>
    </View>
  );
}
```

Stilleri ekleyiniz:

```
ticketCard: {
  backgroundColor: '#ffffff',
  padding: 16,
  borderRadius: 12,
  marginBottom: 12,
```

```
  borderWidth: 1,
  borderColor: '#e2e8f0'
},

ticketTop: {
  flexDirection: 'row',
  justifyContent: 'space-between',
  alignItems: 'center'
},

ticketId: {
  color: '#64748b',
  fontSize: 12
},

ticketTitle: {
  color: '#0f172a',
  fontSize: 17,
  fontWeight: 'bold',
  marginTop: 12
},

ticketDetail: {
  color: '#64748b',
  fontSize: 13,
  marginTop: 6
},

ticketBottom: {
  flexDirection: 'row',
  justifyContent: 'space-between',
  alignItems: 'center',
  marginTop: 16
},

statusText: {
  fontWeight: 'bold',
  fontSize: 13
},

actionButton: {
  backgroundColor: '#dbeafe',
  paddingVertical: 9,
  paddingHorizontal: 12,
  borderRadius: 8
},

actionText: {
```

```
  color: '#1d4ed8',
  fontWeight: '600'
}
```

Bu kart, kendisine verilen `talep` nesnesindeki bilgileri gösterecek şekilde tasarlanmıştır. Böylece her arıza için ayrı bir bileşen kodu yazmak gerekmeyecektir.

Adım 7 — FlatList ile talepleri listeleyiniz

`FlatList`, özellikle çok sayıda kayıt içeren listelerin oluşturulmasında kullanılan React Native bileşenidir.

`ScrollView` içindeki normal bir `map()` kullanımından farklı olarak, liste öğelerini pencereleme (virtualization) yaklaşımıyla yönetir ve uzun listelerde performans açısından avantaj sağlayabilir.

React Native
+1

Temel kullanımı:

```
<FlatList
  data={talepler}
  keyExtractor={(item) => item.id}
  renderItem={({ item }) => (
    <TalepKarti
      talep={item}
      onDurumDegistir={durumDegistir}
    />
  )}
/>
```

Önemli özellikler:

Özellik	İşlevi
`data`	Listelenecek veri dizisi
`renderItem`	Her kayıt için oluşturulacak bileşen
`keyExtractor`	Her öğenin benzersiz anahtarı
`ListHeaderComponent`	Liste üstünde gösterilecek içerik

`ListEmptyComponent`	Liste boş olduğunda gösterilecek bileşen
`contentContainerStyle`	Liste içeriğinin yerleşim stili

Tasarım kararı: Yönetim panelindeki başlık, özet kartları ve filtreler de kaydırılabilir alanın içerisinde bulunacaktır. Bu nedenle bunları `FlatList` bileşeninin `ListHeaderComponent` özelliğiyle oluşturacağız.

Ayrıca tüm ekranı ayrı bir `ScrollView` ile sararak iç içe dikey kaydırılabilir listeler oluşturmayacağız.

Adım 8 — Durum filtrelerini geliştiriniz

Müşteri aşağıdaki filtreleri istemektedir:

`Tümü`, `Açık`, `İşlemde`, `Tamamlandı`

Öncelikle filtre dizisini tanımlayınız:

```
const filtreler = [
  'Tümü',
  'Açık',
  'İşlemde',
  'Tamamlandı'
];
```

`TeknikDestekPaneli` fonksiyonunda filtrelenmiş veriyi hesaplayınız:

```
const filtreliTalepler = talepler.filter((item) => {
  if (aktifFiltre === 'Tümü') {
    return true;
  }

  return item.durum === aktifFiltre;
});
```

Filtre butonları:

```
<View style={styles.filterContainer}>
  {filtreler.map((filtre) => (
    <Pressable
      key={filtre}
      onPress={() => setAktifFiltre(filtre)}
      style={[
        styles.filterButton,
        aktifFiltre === filtre && styles.activeFilter
```

```
      ]}
    >
      <Text
        style={[
          styles.filterText,
          aktifFiltre === filtre && styles.activeFilterText
        ]}
      >
        {filtre}
      </Text>
    </Pressable>
  ))}
</View>
```

Stiller:

```
filterContainer: {
  flexDirection: 'row',
  flexWrap: 'wrap',
  gap: 8,
  marginTop: 15,
  marginBottom: 15
},

filterButton: {
  backgroundColor: '#e2e8f0',
  paddingVertical: 10,
  paddingHorizontal: 12,
  borderRadius: 9
},

activeFilter: {
  backgroundColor: '#2563eb'
},

filterText: {
  color: '#475569',
  fontSize: 13,
  fontWeight: '600'
},

activeFilterText: {
  color: '#ffffff'
}
```

`FlatList` bileşenindeki veri kaynağını `filtreliTalepler` olarak değiştiriniz:

```
data={filtreliTalepler}
```

Kontrol: Açık filtresine basıldığında başlangıçta yalnızca üç açık talep görüntülenmelidir. Toplam özet kartı ise bütün kayıtların sayısını göstermeye devam etmelidir.

Adım 9 — Talep durumlarını güncelleyiniz

Her destek talebinin üç aşamalı bir iş akışı bulunmaktadır:

Açık

İşlemde

Tamamlandı

Bir talebin durum değişimi

Kullanıcı İşleme Al butonuna bastığında Açık talep İşlemde durumuna geçecektir. Tamamla butonuna basıldığında İşlemde talep Tamamlandı durumuna geçecektir.

`TeknikDestekPaneli` içerisinde aşağıdaki fonksiyonu yazınız:

```
const durumDegistir = (id) => {
  setTalepler((oncekiTalepler) =>
    oncekiTalepler.map((item) => {

      if (item.id !== id) {
        return item;
      }

      let yeniDurum = item.durum;

      if (item.durum === 'Açık') {
        yeniDurum = 'İşlemde';
      } else if (item.durum === 'İşlemde') {
        yeniDurum = 'Tamamlandı';
      }

      return {
        ...item,
        durum: yeniDurum
      };
    })
  );
};
```

Kodun çalışma mantığı:

- `map()` mevcut talep dizisinden yeni bir dizi oluşturur.
- `item.id !== id` koşulu, değiştirilmeyecek talepleri olduğu gibi döndürür.
- İlgili talep için yeni bir nesne oluşturulur.

- `...item` ifadesi mevcut alanları yeni nesneye kopyalar.
- `durum` alanına yeni değer atanır.
- `setTalepler()` ile uygulama durumu güncellenir.

React durumunu güncellerken mevcut diziyi veya nesneyi doğrudan değiştirmek yerine yeni bir dizi ve nesne oluşturuyoruz.

Adım 10 — Responsive özet kartları oluşturunuz

Geliştirdiğimiz panelin farklı ekran genişliklerinde kullanılabilmesi gerekir.

React Native'in `useWindowDimensions` hook'u, kullanılabilir pencere boyutlarını sağlar ve boyutlar değiştiğinde bileşenin güncellenmesine yardımcı olur.

Önce şu ifadeyi ekleyiniz:

const { width } = useWindowDimensions();

Ekran genişliğine göre özet kartlarının sütun sayısını hesaplayınız:

const sutunSayisi = width >= 600 ? 4 : 2;

Bu laboratuvarda aşağıdaki kuralı uygulayacağız:

- 600 dp altındaki ekranlarda iki sütun
- 600 dp ve üzerindeki ekranlarda dört sütun

`dp` cihaz yoğunluğundan bağımsız yerleşim birimi olarak düşünülebilir.

Kart genişliğini hesaplayınız:

```
const kartBosluk = 10;
const yatayBosluk = 40;

const kartGenisligi =
  (width - yatayBosluk - (sutunSayisi - 1) * kartBosluk)
  / sutunSayisi;
```

Böylece özet kartlarının sabit genişlik yerine kullanılabilir alana göre boyutlandırılması sağlanır.

Önemli: Bu hesaplama, ekranın sağında ve solunda toplam 40 dp dış boşluk ve kartlar arasında 10 dp boşluk kullanıldığı varsayımıyla yapılmıştır.

Tam kaynak kodda bu değerleri birlikte kullanarak responsive kart düzenini tamamlayacağız.

8. Uygulamanın Tam ve Birleştirilmiş Kaynak Kodu

Aşağıdaki kod, laboratuvarda geliştirilen bütün özellikleri içeren uygulamanın son hâlidir.

Bu sürümde özet kartları, ekran genişliğine göre iki veya dört sütunda görüntülenir. Listeleme `FlatList` ile gerçekleştirilir. Talep kartları yeniden kullanılabilir bileşenlerden oluşturulur.

Dosya: `App.js`

```
import React, { useState } from 'react';

import {
  View,
  Text,
  Pressable,
  FlatList,
  StyleSheet,
  StatusBar,
  useWindowDimensions
} from 'react-native';

import {
  SafeAreaProvider,
  SafeAreaView
} from 'react-native-safe-area-context';

// 1. ÖRNEK TALEP VERİLERİ

const baslangicTalepleri = [
  {
    id: 'TLP-1001',
    baslik: 'Bilgisayar açılmıyor',
    kategori: 'Bilgisayar',
    birim: 'Öğrenci İşleri',
    oncelik: 'Yüksek',
    durum: 'Açık'
  },
  {
    id: 'TLP-1002',
    baslik: 'Yazıcı bağlantı sorunu',
    kategori: 'Yazıcı',
    birim: 'Personel Daire Başkanlığı',
    oncelik: 'Orta',
    durum: 'İşlemde'
  },
  {
    id: 'TLP-1003',
    baslik: 'İnternet bağlantısı kesildi',
    kategori: 'İnternet',
    birim: 'Kütüphane',
    oncelik: 'Yüksek',
```

```
    durum: 'Açık'
  },
  {
    id: 'TLP-1004',
    baslik: 'Ofis yazılımı hata veriyor',
    kategori: 'Yazılım',
    birim: 'Muhasebe',
    oncelik: 'Düşük',
    durum: 'Tamamlandı'
  },
  {
    id: 'TLP-1005',
    baslik: 'Bilgisayar çok yavaş',
    kategori: 'Bilgisayar',
    birim: 'Mühendislik Fakültesi',
    oncelik: 'Orta',
    durum: 'İşlemde'
  },
  {
    id: 'TLP-1006',
    baslik: 'Yazıcı kağıt sıkıştırıyor',
    kategori: 'Yazıcı',
    birim: 'Rektörlük',
    oncelik: 'Düşük',
    durum: 'Tamamlandı'
  },
  {
    id: 'TLP-1007',
    baslik: 'Kablosuz ağ çalışmıyor',
    kategori: 'İnternet',
    birim: 'Merkezi Derslikler',
    oncelik: 'Yüksek',
    durum: 'Açık'
  },
  {
    id: 'TLP-1008',
    baslik: 'Program lisans sorunu',
    kategori: 'Yazılım',
    birim: 'Bilgi İşlem',
    oncelik: 'Orta',
    durum: 'İşlemde'
  }
];

// 2. SABİT SEÇENEKLER

const filtreler = [
  'Tümü',
```

```
  'Açık',
  'İşlemde',
  'Tamamlandı'
];

// 3. RENK YARDIMCI FONKSİYONLARI

const oncelikRengi = (oncelik) => {
  if (oncelik === 'Yüksek') return '#dc2626';
  if (oncelik === 'Orta') return '#d97706';
  return '#16a34a';
};

const durumRengi = (durum) => {
  if (durum === 'Açık') return '#dc2626';
  if (durum === 'İşlemde') return '#2563eb';
  return '#16a34a';
};

// 4. ÖZET KARTI BİLEŞENİ

function OzetKarti({
  baslik,
  sayi,
  renk,
  genislik
}) {
  return (
    <View
      style={[
        styles.summaryCard,
        { width: genislik }
      ]}
    >
      <View
        style={[
          styles.summaryIndicator,
          { backgroundColor: renk }
        ]}
      />

      <Text
        style={[
          styles.summaryNumber,
          { color: renk }
        ]}
      >
        {sayi}
```

```
      </Text>

      <Text style={styles.summaryLabel}>
        {baslik}
      </Text>
    </View>
  );
}

// 5. TALEP KARTI BİLEŞENİ

function TalepKarti({
  talep,
  onDurumDegistir
}) {
  const tamamlandi = talep.durum === 'Tamamlandı';

  return (
    <View style={styles.ticketCard}>

      {/* ÜST SATIR */}
      <View style={styles.ticketTop}>

        <Text style={styles.ticketId}>
          {talep.id}
        </Text>

        <View
          style={[
            styles.priorityBadge,
            {
              backgroundColor:
                oncelikRengi(talep.oncelik) + '18'
            }
          ]}
        >
          <Text
            style={[
              styles.priorityText,
              { color: oncelikRengi(talep.oncelik) }
            ]}
          >
            {talep.oncelik}
          </Text>
        </View>

      </View>
```

```
{/* ARIZA BAŞLIĞI */}
<Text style={styles.ticketTitle}>
  {talep.baslik}
</Text>

{/* KATEGORİ VE BİRİM */}
<Text style={styles.ticketDetail}>
  {talep.kategori} • {talep.birim}
</Text>

{/* ALT SATIR */}
<View style={styles.ticketBottom}>

  <View
    style={[
      styles.statusBadge,
      {
        backgroundColor:
          durumRengi(talep.durum) + '18'
      }
    ]}
  >
    <Text
      style={[
        styles.statusText,
        { color: durumRengi(talep.durum) }
      ]}
    >
      {talep.durum}
    </Text>
  </View>

  {!tamamlandi && (
    <Pressable
      style={styles.actionButton}
      onPress={() => onDurumDegistir(talep.id)}
    >
      <Text style={styles.actionText}>
        {talep.durum === 'Açık'
          ? 'İşleme Al →'
          : 'Tamamla ✓'}
      </Text>
    </Pressable>
  )}

  {tamamlandi && (
    <Text style={styles.completedText}>
      İşlem tamamlandı
```

```
          </Text>
        )}

      </View>
    </View>
  );
}

// 6. ANA YÖNETİM PANELİ

function TeknikDestekPaneli() {

  // STATE
  const [talepler, setTalepler] = useState(
    baslangicTalepleri
  );

  const [aktifFiltre, setAktifFiltre] = useState('Tümü');

  // EKRAN BOYUTLARI
  const { width } = useWindowDimensions();

  // ÖZET KARTI YERLEŞİMİ
  const sutunSayisi = width >= 600 ? 4 : 2;

  const kartGenisligi =
    (width - 40 - (sutunSayisi - 1) * 10)
    / sutunSayisi;

  // TALEP SAYILARI
  const toplam = talepler.length;

  const acik = talepler.filter(
    item => item.durum === 'Açık'
  ).length;

  const islemde = talepler.filter(
    item => item.durum === 'İşlemde'
  ).length;

  const tamamlanan = talepler.filter(
    item => item.durum === 'Tamamlandı'
  ).length;

  // 7. FİLTRELEME

  const filtreliTalepler = talepler.filter((item) => {
    if (aktifFiltre === 'Tümü') {
```

```
    return true;
  }

  return item.durum === aktifFiltre;
});

// 8. TALEP DURUMU GÜNCELLEME

const durumDegistir = (id) => {

  setTalepler((oncekiTalepler) =>
    oncekiTalepler.map((item) => {

      if (item.id !== id) {
        return item;
      }

      let yeniDurum = item.durum;

      if (item.durum === 'Açık') {
        yeniDurum = 'İşlemde';
      } else if (item.durum === 'İşlemde') {
        yeniDurum = 'Tamamlandı';
      }

      return {
        ...item,
        durum: yeniDurum
      };
    })
  );
};

// 9. LİSTE ÜST ALANI

const listeBasligi = (
  <View>

    {/* PANEL BAŞLIĞI */}
    <View style={styles.header}>

      <Text style={styles.headerLabel}>
        BİLGİ İŞLEM DAİRE BAŞKANLIĞI
      </Text>

      <Text style={styles.title}>
        Teknik Destek Paneli
      </Text>
```

```
  <Text style={styles.subtitle}>
    Destek taleplerini buradan yönetin.
  </Text>

</View>

{/* ÖZET KARTLARI */}
<View style={styles.summaryGrid}>

  <OzetKarti
    baslik="Toplam Talep"
    sayi={toplam}
    renk="#2563eb"
    genislik={kartGenisligi}
  />

  <OzetKarti
    baslik="Açık Talep"
    sayi={acik}
    renk="#dc2626"
    genislik={kartGenisligi}
  />

  <OzetKarti
    baslik="İşlemde"
    sayi={islemde}
    renk="#d97706"
    genislik={kartGenisligi}
  />

  <OzetKarti
    baslik="Tamamlanan"
    sayi={tamamlanan}
    renk="#16a34a"
    genislik={kartGenisligi}
  />

</View>

{/* TALEP LİSTESİ BAŞLIĞI */}
<View style={styles.sectionHeader}>

  <Text style={styles.sectionTitle}>
    Destek Talepleri
  </Text>

  <Text style={styles.sectionCount}>
```

```
        {filtreliTalepler.length} kayıt
      </Text>

    </View>

    {/* FİLTRELER */}
    <View style={styles.filterContainer}>

      {filtreler.map((filtre) => (

        <Pressable
          key={filtre}
          onPress={() => setAktifFiltre(filtre)}
          style={[
            styles.filterButton,
            aktifFiltre === filtre &&
              styles.activeFilter
          ]}
        >
          <Text
            style={[
              styles.filterText,
              aktifFiltre === filtre &&
                styles.activeFilterText
            ]}
          >
            {filtre}
          </Text>
        </Pressable>

      ))}

    </View>

  </View>
);

// 10. EKRAN ÇIKTISI

return (
  <SafeAreaView style={styles.container}>

    <StatusBar
      barStyle="dark-content"
      backgroundColor="#f1f5f9"
    />

    <FlatList
```

```
        data={filtreliTalepler}

        keyExtractor={(item) => item.id}

        renderItem={({ item }) => (
          <TalepKarti
            talep={item}
            onDurumDegistir={durumDegistir}
          />
        )}

        ListHeaderComponent={listeBasligi}

        ListEmptyComponent={
          <View style={styles.emptyContainer}>
            <Text style={styles.emptyTitle}>
              Kayıt bulunamadı
            </Text>
            <Text style={styles.emptyText}>
              Seçilen duruma ait destek talebi yok.
            </Text>
          </View>
        }

        contentContainerStyle={styles.listContent}

        showsVerticalScrollIndicator={false}
      />

    </SafeAreaView>
  );
}

// 11. UYGULAMA KÖKÜ

export default function App() {
  return (
    <SafeAreaProvider>
      <TeknikDestekPaneli />
    </SafeAreaProvider>
  );
}

// 12. STİLLER

const styles = StyleSheet.create({

  container: {
```

```
  flex: 1,
  backgroundColor: '#f1f5f9'
},

listContent: {
  paddingHorizontal: 20,
  paddingBottom: 45,
  flexGrow: 1
},

// BAŞLIK
header: {
  backgroundColor: '#dbeafe',
  borderRadius: 16,
  padding: 20,
  marginTop: 12,
  marginBottom: 18
},

headerLabel: {
  fontSize: 11,
  fontWeight: 'bold',
  color: '#1d4ed8',
  letterSpacing: 0.8
},

title: {
  fontSize: 25,
  fontWeight: 'bold',
  color: '#0f172a',
  marginTop: 9
},

subtitle: {
  fontSize: 14,
  color: '#475569',
  marginTop: 8
},

// ÖZET KARTLARI
summaryGrid: {
  flexDirection: 'row',
  flexWrap: 'wrap',
  gap: 10,
  marginBottom: 25
},

summaryCard: {
```

```
  backgroundColor: '#ffffff',
  borderRadius: 13,
  padding: 16,
  minHeight: 108,
  justifyContent: 'center',
  borderWidth: 1,
  borderColor: '#e2e8f0'
},

summaryIndicator: {
  width: 28,
  height: 4,
  borderRadius: 2,
  marginBottom: 9
},

summaryNumber: {
  fontSize: 29,
  fontWeight: 'bold'
},

summaryLabel: {
  color: '#64748b',
  fontSize: 13,
  marginTop: 5
},

// LİSTE BAŞLIĞI
sectionHeader: {
  flexDirection: 'row',
  justifyContent: 'space-between',
  alignItems: 'center'
},

sectionTitle: {
  fontSize: 20,
  fontWeight: 'bold',
  color: '#0f172a',
  flexShrink: 1
},

sectionCount: {
  fontSize: 13,
  color: '#64748b'
},

// FİLTRELER
filterContainer: {
```

```
  flexDirection: 'row',
  flexWrap: 'wrap',
  gap: 8,
  marginTop: 15,
  marginBottom: 18
},

filterButton: {
  backgroundColor: '#e2e8f0',
  paddingVertical: 10,
  paddingHorizontal: 13,
  borderRadius: 9
},

activeFilter: {
  backgroundColor: '#2563eb'
},

filterText: {
  color: '#475569',
  fontSize: 13,
  fontWeight: '600'
},

activeFilterText: {
  color: '#ffffff'
},

// TALEP KARTLARI
ticketCard: {
  backgroundColor: '#ffffff',
  padding: 16,
  borderRadius: 13,
  marginBottom: 12,
  borderWidth: 1,
  borderColor: '#e2e8f0'
},

ticketTop: {
  flexDirection: 'row',
  justifyContent: 'space-between',
  alignItems: 'center',
  gap: 10
},

ticketId: {
  color: '#64748b',
  fontSize: 12
```

```
},

priorityBadge: {
  paddingHorizontal: 11,
  paddingVertical: 6,
  borderRadius: 20
},

priorityText: {
  fontSize: 12,
  fontWeight: 'bold'
},

ticketTitle: {
  color: '#0f172a',
  fontSize: 17,
  fontWeight: 'bold',
  marginTop: 12
},

ticketDetail: {
  color: '#64748b',
  fontSize: 13,
  marginTop: 6
},

ticketBottom: {
  flexDirection: 'row',
  justifyContent: 'space-between',
  alignItems: 'center',
  marginTop: 16,
  gap: 10
},

statusBadge: {
  paddingHorizontal: 11,
  paddingVertical: 7,
  borderRadius: 8
},

statusText: {
  fontSize: 12,
  fontWeight: 'bold'
},

actionButton: {
  backgroundColor: '#dbeafe',
  paddingVertical: 10,
```

```
    paddingHorizontal: 12,
    borderRadius: 8
  },

  actionText: {
    color: '#1d4ed8',
    fontSize: 12,
    fontWeight: 'bold'
  },

  completedText: {
    fontSize: 12,
    color: '#16a34a',
    fontWeight: '600'
  },

  // BOŞ LİSTE
  emptyContainer: {
    alignItems: 'center',
    padding: 30,
    marginTop: 10,
    backgroundColor: '#ffffff',
    borderRadius: 12
  },

  emptyTitle: {
    fontSize: 17,
    fontWeight: 'bold',
    color: '#334155'
  },

  emptyText: {
    fontSize: 13,
    color: '#64748b',
    marginTop: 8,
    textAlign: 'center'
  }

});
```

8.1. Tam kodun çalıştırılması

Kodun tamamını `App.js` dosyasına yerleştiriniz.

Terminalde:

npx expo start

komutunu çalıştırınız.

Expo Go ile uygulamayı açınız.

Ekranda aşağıdaki bileşenlerin bulunması beklenmektedir:

- Teknik Destek Paneli başlığı
- Dört özet bilgi kartı
- Durum filtreleme butonları
- Sekiz başlangıç talebi
- Her talebin öncelik ve durum bilgileri
- Durum güncelleme butonları

Deneme: `TLP-1001` numaralı talepte İşleme Al butonuna basınız. Talep durumu `İşlemde` olmalı ve özet kartlarındaki sayılar değişmelidir.

Başlangıçta Açık talep sayısı 3 iken bu işlemden sonra 2; İşlemde talep sayısı ise 3 iken 4 olmalıdır.

Ardından aynı talep için Tamamla butonuna basarak durumun `Tamamlandı` olarak güncellendiğini kontrol ediniz.

Filtre aktifken bir talebin durumu değiştirildiğinde, yeni durumu seçili filtreyle eşleşmiyorsa kartın listeden kaybolması beklenen davranıştır. Talep silinmez; farklı filtre altında görüntülenir.

9. Uygulama Testleri

Uygulamanın sadece görsel olarak değil, işlevsel açıdan da doğru çalıştığını kontrol ediniz.

Test Kontrol Listesi

0/10

T01 — Uygulamayı başlatınız.

Dört özet kartı görünmeli; sayılar 8, 3, 3 ve 2 olmalı.

T02 — Tümü filtresini seçiniz.

Sekiz destek talebi görünmeli.

T03 — Açık filtresini seçiniz.

Üç açık talep görünmeli.

T04 — TLP-1001 talebini İşleme Al ile güncelleyiniz.

Talep İşlemde durumuna geçmeli; Açık sayısı 2, İşlemde sayısı 4 olmalı.

T05 — İşlemde filtresinden TLP-1001 talebini tamamlayınız.

Talep Tamamlandı durumuna geçmeli ve özet kartları güncellenmeli.

T06 — Tamamlandı filtresini seçiniz.

Tamamlanan talepler görüntülenmeli.

T07 — Tamamlanmış bir kartı inceleyiniz.

Durum değiştirme butonu bulunmamalı.

T08 — Öncelik renklerini inceleyiniz.

Düşük yeşil, Orta turuncu, Yüksek kırmızı olmalı.

T09 — Cihazı döndürünüz veya farklı ekran genişliği deneyiniz.

Özet kartları ekran genişliğine uyum sağlamalı.

T10 — Bütün uygun talepleri tamamlayıp Açık filtresini seçiniz.

Kayıt bulunamadı mesajı görüntülenmeli.

Test sonuçlarını kopyala

10. Bağımsız Geliştirme Görevleri

Bu bölümde öğrenciler, bir yazılım şirketinin müşterisinden gelen yeni talepleri gerçekleştireceklerdir.

Hazır kod verilmemiştir. Öğrencilerin ders kapsamında öğrendikleri bileşenleri ve JavaScript yöntemlerini kullanmaları beklenmektedir.

Görev 1 — Öncelik filtresi (Orta)

Müşteri, yalnızca talep durumuna göre filtreleme yapılmasını yeterli bulmamaktadır.

Yeni gereksinim: Kullanıcılar talepleri öncelik seviyelerine göre de filtreleyebilmelidir.

Beklenen geliştirmeler:

- `Tümü`, `Düşük`, `Orta`, `Yüksek` öncelik filtrelerini oluşturunuz.
- Durum filtresi ile öncelik filtresi birlikte çalışabilmelidir.
- Örneğin `Açık` ve `Yüksek` seçildiğinde yalnızca bu iki koşulu sağlayan talepler görüntülenmelidir.
- Filtreler değiştiğinde listelenen kayıt sayısı güncellenmelidir.

Görev 2 — Talep detay kartı (Orta–Zor)

Müşteri, her talep hakkında daha fazla bilgi görüntülemek istemektedir.

Yeni gereksinim: Kullanıcı bir talep kartına dokunduğunda ayrıntılı bilgileri görebilmelidir.

Beklenen geliştirmeler:

- Talep kartına basıldığında detaylar gösterilmelidir.
- Ayrıntı alanında talep numarası, arıza başlığı, kategori, birim, öncelik ve durum bulunmalıdır.
- Detay alanı tekrar dokunulduğunda kapanabilmelidir.
- Bunun için yeni ekran veya navigasyon kütüphanesi kullanılmamalıdır.

İpucu: `useState` ve koşullu render kullanılabilir.

Görev 3 — Önceliğe göre sıralama (Zor)

Müşteri, en kritik talepleri listenin üst kısmında görmek istemektedir.

Yeni gereksinim: Talepler öncelik seviyesine göre sıralanmalıdır.

Sıralama şu şekilde olmalıdır:

1. Yüksek öncelikli talepler
2. Orta öncelikli talepler
3. Düşük öncelikli talepler

Öğrenci, `sort()` fonksiyonu kullanarak bir sıralama mekanizması geliştirmelidir.

Önemli: React state içerisindeki mevcut dizi doğrudan değiştirilmemelidir. Sıralama işlemi dizinin kopyası üzerinden yapılmalıdır.

Görev 4 — Yönetim paneli tasarımını geliştirme (Zor)

Müşteri, uygulamanın daha profesyonel bir görünüme sahip olmasını istemektedir.

Aşağıdaki geliştirmeleri gerçekleştiriniz:

- Talep kategorilerini temsil eden simgeler ekleyiniz.
- Özet kartlarında simgeler kullanınız.
- Kartlar arasında tutarlı boşluklar oluşturunuz.
- Seçilen filtrelerin görünürlüğünü artırınız.
- Küçük ekranlarda taşma ve üst üste binme sorunlarını gideriniz.
- Uygulamanın görsel hiyerarşisini iyileştiriniz.

Simgeler için Expo ile uyumlu `@expo/vector-icons` kullanılabilir. Öğrenciler bu görevde ilgili kütüphanenin kullanımını araştırmalıdır.

11. Laboratuvar Tamamlama ve Teslim

Öğrenciler, hazırladıkları Teknik Destek Yönetim Paneli uygulamasının çalıştığını gösteren kanıtları Google Form üzerinden teslim edeceklerdir.

Önerilen form alanları:

1. Öğrenci numarası, ad ve soyad.
2. Yönetim panelinin ilk açılış ekran görüntüsü.
3. Açık taleplerin filtrelendiği ekran görüntüsü.
4. İşleme alınan bir talebin güncellenmiş ekran görüntüsü.
5. Responsive kart düzeninin ekran görüntüsü.
6. Tamamlanan bağımsız görevlerin seçilmesi.
7. Yapılan geliştirmelerin kısa açıklaması.
8. Projenin GitHub bağlantısı (isteniyorsa).

12. Laboratuvar Sonu Değerlendirme Soruları

1. React Native'de `flexDirection: 'row'` ile `flexDirection: 'column'` arasındaki fark nedir?
2. `justifyContent` ve `alignItems` özelliklerinin görevleri nelerdir?
3. `FlatList` kullanmanın uzun veri listelerinde avantajları nelerdir?
4. `keyExtractor` özelliği neden gereklidir?
5. `OzetKarti` ve `TalepKarti` bileşenlerini ayrı fonksiyonlar olarak oluşturmanın avantajı nedir?
6. `filter()` fonksiyonu ile listeleme işlemi nasıl gerçekleştirilmektedir?
7. `map()` fonksiyonu ile talep durumlarını güncellerken neden yeni nesne oluşturulmaktadır?
8. `useWindowDimensions` responsive tasarım açısından ne sağlar?
9. Bir talebin durumu değiştirildiğinde özet kartlarındaki sayıların otomatik güncellenmesini sağlayan mekanizma nedir?
10. Gerçek bir kurumda kullanılacak sistem için bu prototipe hangi özellikler eklenmelidir?

13. Laboratuvar Çalışmasının Sonucu

Bu laboratuvarda öğrenciler, React Native kullanarak teknik destek taleplerini görüntüleyen ve yöneten bir mobil uygulama arayüzü geliştirmişlerdir.

Önceki lab föyünde veri giriş odaklı uygulamasından farklı olarak bu çalışmada veri sunumu, kart düzeni, listeleme, filtreleme, durum güncelleme ve responsive tasarım üzerinde durulmuştur.

İki haftalık çalışmaların birlikte değerlendirilmesiyle, kurumsal bir teknik destek sisteminin iki temel kullanım senaryosu ele alınmış olacaktır:

Lab Föy 1: Personel arıza bildirimi oluşturur.

Lab Föy 2: Teknik destek personeli talepleri görüntüler ve durumlarını yönetir.

Bu iki uygulama şimdilik bağımsız prototiplerdir; aynı veritabanını paylaşmamaktadır. İlerleyen haftalarda API entegrasyonu, kalıcı veri yönetimi ve ekranlar arası navigasyon konuları işlendiğinde birleştirilmeye uygun bir temel oluştururlar.

Teknik dokümantasyon: React Native — Flexbox, React Native — FlatList ve Expo — Safe Areas.